Planejamento de Etapas

Etapa 1: Unificar o núcleo

Sócio Digital: Copiloto de Gestão Fiscal para Pequenos Negócios

Objetivo

Ligar o RAG (backend/api) ao motor fiscal determinístico (src/fiscal) e criar o modelo de dados da empresa. Ao final, a API responde com base na lei e no cálculo, a partir do perfil da empresa, em um plano de 3 passos (RF01, RF03, RF07).

Ponto de partida

Hoje o repositório tem duas pilhas paralelas: src/ (RAG antigo em SQLite e motor fiscal) e backend/api/ (RAG novo com Qdrant e Ollama). O motor fiscal tem testes, mas não é exposto pela API, e não existe cadastro de empresa.

Resultado esperado

API única com endpoints fiscais, perfil da empresa persistido e assistente que combina norma e cálculo.

Resumo das issues

ID	Issue	Prioridade	Responsável sugerido
E1.1	Migrar o motor fiscal de src/fiscal para backend	Alta	Lucas
E1.2	Endpoint POST /fiscal/simular	Alta	Lucas
E1.3	Modelo de dados do perfil da empresa (PostgreSQL)	Alta	Lucas, com Clara e Nasser
E1.4	API de cadastro e consulta da empresa	Alta	Lucas
E1.5	Consulta de CNPJ com preenchimento manual como alternativa	Média	Lucas
E1.6	Orquestrar RAG e cálculo no assistente (plano em 3 passos)	Alta	Jhessica e Lucas
E1.7	Aposentar src/rag e consolidar os testes da raiz	Média	Jhessica

Cada issue abaixo está escrita para ser copiada ao GitHub: o título da seção vira o título da issue; os rótulos indicados devem ser criados no repositório; a descrição e os critérios de aceite vão no corpo. Responsáveis e prioridades são sugestões e devem ser confirmados pela equipe.

Issues

[E1.1] Migrar o motor fiscal de src/fiscal para backend

Responsável sugerido: Lucas | **Prioridade:** Alta

Rótulos: etapa-1, backend, refactor | **Depende de:** Nenhuma

Descrição

Mover calculator, models, repository e ingest de src/fiscal para um pacote dentro de backend/, mantendo as tabelas do Simples (30 faixas) e a base de CNAEs. Ajustar imports, o CI e os testes de raiz (test_fiscal.py).

Critérios de aceite

- Código fiscal fica em backend/, sem importar de src/
- Testes do cálculo migrados e passando no CI
- Dados de tabelas e CNAEs carregáveis por comando documentado

---

[E1.2] Endpoint POST /fiscal/simular

Responsável sugerido: Lucas | **Prioridade:** Alta

Rótulos: etapa-1, backend, feature | **Depende de:** Migrar o motor fiscal

Descrição

Expor o cálculo do Simples Nacional: recebe RBT12, CNAE e folha (opcional) e devolve alíquota efetiva, anexo aplicado, Fator R, tributos segregados e alerta de desenquadramento do MEI (limite de faturamento).

Critérios de aceite

- Schemas de entrada e saída validados (422 em dados inválidos)
- Resultado idêntico ao do calculator nos casos de teste existentes
- Documentado no Swagger com exemplos
- Testes de API cobrindo MEI, Fator R acima e abaixo de 28% e CNAE inexistente

---

[E1.3] Modelo de dados do perfil da empresa (PostgreSQL)

Responsável sugerido: Lucas, com Clara e Nasser | **Prioridade:** Alta

Rótulos: etapa-1, backend, dados | **Depende de:** Nenhuma

Descrição

Criar o esquema da empresa: CNPJ, CNAE principal, município/UF, porte, regime e enquadramento, faturamento e folha. Cada campo registra a origem (informado pelo usuário ou consultado) e a data. Reaproveitar o que for útil do contrato SQL do refino.

Critérios de aceite

- Migrações versionadas (ex.: Alembic)
- Campo de origem por dado, como no wireframe da tela 02
- Compose com o serviço de banco para desenvolvimento

[E1.4] API de cadastro e consulta da empresa

Responsável sugerido: Lucas | **Prioridade:** Alta

Rótulos: etapa-1, backend, feature | **Depende de:** Modelo de dados do perfil

Descrição

Endpoints para criar, ler e atualizar o perfil da empresa, com validação de CNPJ e de CNAE contra a base. Preparar o vínculo com o usuário para a etapa 3.

Critérios de aceite

- CRUD funcional com validação de CNPJ (dígitos verificadores)
- Erros por campo, como previsto no wireframe
- Testes automatizados

[E1.5] Consulta de CNPJ com preenchimento manual como alternativa

Responsável sugerido: Lucas | **Prioridade:** Média

Rótulos: etapa-1, backend, integração | **Depende de:** API de cadastro da empresa

Descrição

Integrar uma fonte pública de consulta de CNPJ para pré-preencher o cadastro. Se a consulta estiver indisponível ou falhar, o fluxo manual continua funcionando. Verificar termos de uso e limites da fonte escolhida antes de integrar.

Critérios de aceite

- Retorno exibido para conferência, com origem marcada
- Timeout e erro tratados sem bloquear o cadastro
- Fonte e limites documentados

[E1.6] Orquestrar RAG e cálculo no assistente (plano em 3 passos)

Responsável sugerido: Jhessica e Lucas | **Prioridade:** Alta

Rótulos: etapa-1, backend, ia | **Depende de:** Endpoint /fiscal/simular; API de cadastro da empresa

Descrição

Estender /rag/query (ou criar /assistente/consultar) para receber o perfil da empresa, detectar quando há cálculo a fazer, chamar o motor fiscal e montar a resposta com norma citada, resultado do cálculo e plano em 3 etapas (RF07). Reaproveitar o extrator de entidades de src/rag.

Critérios de aceite

- Resposta separa claramente o que vem da norma e o que vem do cálculo
- Números nunca são gerados pelo modelo: vêm do motor fiscal
- Plano em 3 passos presente quando aplicável
- Casos com e sem perfil de empresa testados

[E1.7] Aposentar src/rag e consolidar os testes da raiz

Responsável sugerido: Jhessica	**Prioridade:** Média
Rótulos: etapa-1, refactor, dívida-técnica	**Depende de:** Orquestração do assistente

Descrição

Portar para backend/ o que ainda vale do RAG antigo (extrator de entidades, prompts e testes de governança) e remover src/rag, o SQLite vetorial e os test_*.py soltos na raiz. Atualizar a documentação em docs/projeto/status-entregas-equipe.md.

Critérios de aceite

- Nenhum código ativo depende de src/rag
- Cenários de governança e abstenção preservados como testes do backend
- README e documentação atualizados